

个人简历

姓名：王剑涛 性别：男
学历：本科 电话号码：17386833425
邮箱：17386833425@163.com 籍贯：吉林省长春市

个人技能

了解AI Agent技术，理解其核心原理，能够利用Dify平台进行多智能体协作系统的设计与开发。
了解MCP协议、A2A通信协议及Function Call架构
熟练应用COT、TOT、Reflexion等高级提示词技术
了解LoRA、QLoRA等大模型高效微调方法，参与小样本场景下的模型参数高效微调
掌握RAG架构设计与开发流程，参与构建多层级检索系统，通过混合检索、重排序及元数据过滤等多策略优化，提升问题回答的相关性与准确率
掌握 BM25、BGE-M3等检索技术
掌握Transformer架构及Attention机制
熟练使用Mysql、Redis、Milvus等数据库
掌握常见的机器学习和深度学习算法，如决策树、随机森林、RNN、LSTM、GRU等
熟悉常见预训练模型的原理和结构，如BERT及其变体，GPT等
熟悉常见的模型优化方法如量化，蒸馏，剪枝等
熟练使用PyTorch框架完成模型搭建、训练与调试

实习经历

2025-7 ~ 2026-1 沈阳国创人工智能科技有限公司 大模型实习生
参与多智能体子系统的设计与实现，重点负责商品咨询agent模块的实现，同时也参与了子 Agent 的整体创建与运行流程设计。
负责商品咨询agent中RAG系统的全链路诊断与调优，重点解决检索不准、响应慢的问题。具体包括优化文档分块、微调嵌入模型、设计混合检索策略，并建立量化评估体系。
负责AI问答功能的设计与实现，利用大模型API进行智能应用开发与集成，并设计多轮对话逻辑。

项目经验

项目：多agent智能客服 2025-07 ~ 2026-01

项目背景：

这是一个面向电商场景的智能多 Agent 交互系统，主要解决传统电商客服与运营流程中效率低、响应慢、数据分散的痛点。

主要职责：

我主要参与的是多智能体子系统的设计与实现，重点负责商品咨询agent的设计与实现，同时也参与了 Agent 的整体创建与运行流程设计。

基于RAG的商品咨询：

主要介绍：

- 1、知识库构建：整理清洗 PDF/Word/PPT 等文档，构建 40 万 + 条数据的多模态知识库。
- 2、检索架构设计：
 - (1)搭建 Milvus 向量数据库，设计 5 个 collections (活动规则, 产品介绍, 售后服务, 发票规则, 配送相关)实现等知识隔离管理。
 - (2)基于 BERT 实现意图分类（5 类场景），F1-score 达94.76%。
 - (3)通过RAG 全流程设计，实现意图路由 - 检索结果拼接 - Prompt 生成的闭环。
- 3、技术优化：
 - (1)引入 OCR 工具提取图文信息，问题处理策略优化 query 理解，提升模糊查询召回率。
 - (2)实现稀疏 + 稠密向量混合检索，加入 Rerank 模块后 Top-5 相关性提升 35%。
 - (3)采用 LoRA 微调 Qwen2.5-7B 模型，适配该公司产品等。
- 4、多模态处理：
 - (1)OCR 集成：用 PaddleOCR 提取 PDF/PPT 图片和表格文本，确保知识库完整。
 - (2)表格处理：用 tabula-py 将 PDF 表格转为 Markdown 或 JSON，方便 LLM 处理。

Agent 的整体创建与运行流程设计：

主要介绍：

第一步是能力建模和 Agent 定义：

在创建子 Agent 之前，我们会先明确它的职责边界、可调用的 MCP 服务范围，以及它在整个 A2A 网络中的角色定位。

比如商品咨询agent，只关注用户提出关于商品的问题并给出答案，避免 Agent 职责膨胀。

第二步是工具与协议层绑定：

每个 Agent 会绑定一组明确的 MCP 接口能力，例如库存管理agent、商品咨询agent等，同时通过 A2A 协议注册自身的输入输出的说明卡片 schema，确保其他 Agent 可以以结构化方式调用它，而不是依赖自然语言的“隐式理解”。

第三步是意图解析与执行链路构建：

在运行时，系统基于 LangGraph的状态图（StateGraph）架构，由意图识别节点（Agent）对用户或系统下发的指令进行结构化解析，并将其拆解为可执行的任务节点与执行链路：

结构化任务拆解：LangGraph 会将用户指令映射为图中的具体执行节点（如库存查询节点、物流跟踪节点），并自动生成节点间的执行依赖关系（如先校验区域参数，再调用 MCP 服务），而非简单的线性调用链；

数据访问与调用：若任务需要访问底层数据，对应业务节点会自动生成标准化 SQL，并通过 A2A 协议触发预设的 MCP 服务调用边（Edge），LangGraph 会自动管理调用状态（如调用成功 / 失败的分支路由）；

缺失参数追问机制：LangGraph 内置的状态（State）管理能力会实时校验任务参数完整性（如区域、时间窗口等），若检测到关键参数缺失，会自动路由至“追问节点”，并将当前执行状态持久化保存；待用户补充参数后，可直接从断点恢复执行链路，无需重新解析指令，彻底避免一次性失败或错误查询。

第四步是结果回传与协同触发：

在 LangGraph 的状态图架构下，子 Agent 执行完成后，其结构化结果会被自动写入全局状态（State）中，再结合架构图的流程进行后续处理：

1. 结构化结果汇聚到输出池:

所有业务节点（如库存监控、商品咨询、订单处理）的执行结果，会以统一 schema 格式写入架构图底部的「输出结果」模块。

2. 基于状态的协同触发:

LangGraph 会根据预设的边（Edge）路由规则，触发跨 Agent 的协同流程，这种基于状态的触发机制，替代了传统的被动回调，实现了更灵活的跨 Agent 联动。

3. 优化输出与用户反馈:

最终，架构图中的「优化输出 agent」会从「输出结果」池读取所有子 Agent 的结果，进行格式统一和内容润色，再返回给用户。

同时，用户的反馈会被实时回传给「意图识别 agent」，作为后续意图识别和模型优化的依据，形成闭环迭代。

教育背景

白城师范学院

软件工程

2022.09-2026-07

自我评价

诚实敬业、工作认真负责、具备较好团队协作精神。

学习能力突出，具备良好逻辑思维与问题拆解能力。

期待在实战中持续成长，提升技术能力为公司和团队创造更多价值